Zodiac V3 Audit

Gnosis Ltd - Report by Côme du Crest

2026-08-06

Table of contents

- Table of contents
- Zodiac V3 Audit
  - Scope
  - Context
  - Status
  - Legal Information And Disclaimer
- Issues
  - [Med][Fixed] Authentication Signature Manipulation
  - [Gas][Acknowledged] Membership._authenticate() verifies signatures twice
  - [Info][Answered] `Membership._hasActiveMembership()` does not take into account consumptions
  - [Info][Acknowledged] Targets with None clearance are not blocked
  - [Info][Acknowledged] `ConditionEvaluator.evaluate()` misinterprets placeholders
  - [Info][Fixed] Misleading documentation of `Integrity._checkSlice()`
  - [Info][Fixed] Misleading documentation of `Integrity._checkPluck()`
  - [Info][Fixed] Topology.inlinedSize() reverts on non inlined element
  - [Question][Answered] `ConditionEvaluator.__input()` value may unexpectedly be a hash
  - [Question][Answered] Adding 32 bytes to start location
  - [Question][Answered] AbiLocation improper use of dynamic elements
  - [Question][Answered] AbiLocation when encoding is None takes first element

Zodiac V3 Audit

This document presents the findings of a smart contract audit conducted by Côme du Crest for Gnosis Ltd.

Scope

Zodiac-Core

The scope includes all contracts within `gnosisguild/zodiac-core` as of commit `0x11708ac`.

After remarks, the code of `zodiac-core` has been reviewed on commit `0xadeb727` of `PR#35`.

Zodiac-Modifier-Roles

The scope also includes all contracts within `gnosisguild/zodiac-modifier-roles` as of commit `0x3c6f49d` of `PR#393`.

After remarks, the code of `zodiac-modifier-roles` has been reviewed on commits `0xa28ac04` and `0x172723b` of `PR#393`.

Additional fix have been reviewed in `PR#501` as of commit `0x81a09df`.

Context

Code structure

`common/AbiDecoder` Decodes ABI calldata, trusts the encoding is sane

`common/AbiLocation` Gives the location / size within calldata of encoded Condition

`common/AllowanceLoader` Loads allowance from storage and returns `(accruedAllowance, timestampToStore)`

`common/ConsumptionList` Makes shallow (reference) copy of a list of Consumptions and replace/add one element

`common/ImmutableStorage` Uses create2 to deploy a contract storing blob in bytecode

`common/PriceConversion` Queries adapter for price and returns `amount*price/1e18`

`core/evaluate/BitmaskChecker` Applies a bitmask to compare calldata bytes to expected values

core/evaluate/WithinAllowanceChecker Ensures a value is below allowance limit by comparing either with storage `allowances[key]` or input `consumption[i]` if key found

core/evaluate/WithinRatioChecker Ensures a ratio of relative/reference is within certain limits. The index of the values within `pluckedValues` is encoded within `compValue` (among other parameters)

core/evaluate/CustomConditionChecker Calls a custom adapter decoded from `compValue` as a condition checker

core/evaluate/ConditionEvaluator Evaluates a condition depending on its operator returning a `Result` struct with a `status` (OK or different violation/errors), a `violatedNodeIndex` (0 if not violated, index of the condition otherwise), `payloadLocation` (0 if OK, input location otherwise), and a `consumptions` array (empty if violated). For most operators the result `consumptions` is the input `consumptions` array. For ZipSome and ArraySome the consumption array is only modified for the one "some" zip used for passing. The `withinAllowance` operator will increase the consumption by the input value (or add it to the list)

core/serialize/ConditionUnpacker Unpacks condition from bytes. If operator is `EqualToAvatar` it is replaced by `EqualTo`, which is why `EqualToAvatar` is not handled in `ConditionEvaluator`

core/serialize/Topology `ConditionFlat[]` assumes parents are directly followed by all of their children (as before no child has two parents, no loop). None encoding have the inlined size of their first child

core/serialize/ConditionPacker AbiEncoded nodes without compValue get a default leadingBytes of `0x0004` (function selector size). If operator is `EqualTo`, then the first word of compValue gets removed. For pluck operators, the pluck index is the first byte of compValue, maxPluckCount represents the highest pluck index

core/serialize/ConditionLoader Loads condition as buffer from immutable storage and unpacks it with `ConditionUnpacker`

core/serialize/ConditionStorer Stores condition as immutable storage and returns the address packed with `ExecutionOptions.pack()` also verifies integrity of conditions

core/serialize/TypeTree Extracts type trees from flat conditions for use in decoding. Provides a definition of structural nodes as being not encoded as None or EtherValue

core/serialize/Integrity Verifies conditions integrity

core/Storage Defines storage of Zodiac Roles module. Defines the keys to access storage as concatenation of target address and selector

core/Authorization If adapter registered for target+selector, unwraps calldata, then validates transaction through `ConditionEvaluator.evaluate()`. Condition is loaded from `scopeConfig` pointed by `roleKey`

core/Membership Provides functions to authenticate a module as a registered module enabled for a role key

core/Settlement Provides `_persist()` function to update membership and consumption in storage for given consumptions and roleKey

core/Setup Owner can enable/disable modules with a specific roleKey and membership, scope targets and functions and perform allowance updates

periphery/AvatarIsOwnerOfERC721 Custom condition that can be called by `CustomConditionChecker` to verify that `avatar` pointed by module is owner of the ERC721 token specified by `to` and `data`

periphery/EIP712Encoder Converts custom ABI encoded calldata into EIP hash. Assumes input calldata elements are packed to 32 bytes

periphery/SignTypedMessageLib Contract (not library) that sets a message as signed in a safe. There are no authorization checks in place and any message can be set

periphery/MultiSendUnwrapper Unpacks Safe `multiSend()` calldata into a custom array of `UnwrappedTransaction`

types/* Defines enums and structs used throughout the code

Roles Main entrypoint. Has `exec...()` functions to execute transactions on the underlying Safe with authorization. Inherits `RolesStorage`, `Modifier` from `zodiac-core` to manage modules and check module signatures (enforces signer is enabled), `Setup` to manage role and scoped targets, `Membership` for authentication (enforces roleKey is authorized and limits on roleKey), `Authorization` for transaction verification rules (enforces tx follows rules set for roleKey), and `Settlement` to update roleKey/sender consumption

Transaction verification flow

- Module or anyone with a module signature calls `Roles.execTransactionWithRole()`
- it calls `Membership._authenticate(roleKey)` which uses `moduleOnly` from `Modifier` in `zodiac-core` ensuring caller/provided signature is from enabled module `modules[signer] != 0` and in case of signature it is not consumed.
- `Membership._authenticate(roleKey)` also recovers signer and ensures `roles[roleKey].members[module]` is valid (as set by owner)
- `Authorization._authorize()` is called to unwrap (if necessary) the transaction data before calling `_transaction()`
- `_transaction()` loads the `scopeConfig` associated with the roleKey/target/selector, ensures options are correct (call/delegatecall/value) and calls `ConditionEvaluator.evaluate()` with the loaded config from `scopeConfig` and transaction context
- `evaluate()` goes through conditions and evaluate wether the transaction is valid. It also computes the `Consumption[]` array (e.g. consuming an token transfer allowance)
- `Roles` calls `target.execTransactionFromModule()` on the target Safe to execute the transaction

- If successful, `Roles` calls `Settlement._persist()` to register consumptions for the roleKey, sender, and allowanceKey

Configuration flow

Packed conditions integrity is solely enforced off-chain, there is no on-chain verification occurring to prevent the owner of Roles to set buggy conditions

Dynamic types and structs

Throughout the code it is often assumed that the first word of dynamic size elements is the size of the element. This is not true in ABI encoding of dynamic structs. However, structs will always be encoded as `Tuple` and and will not have the `Encoding` type `Dynamic` or `AbiEncoded`. Similarly, static size arrays will be encoded as `Tuple` and the `Array` encoding is reserved for dynamic size arrays. The encoding `AbiEncoded` is thus reserved for `bytes[]` and `strings`.

The `condition[]` array is determining what the data will be decoded as and it is set by the "trusted" owner of the Roles mod.

Validations enforced by Integrity

The integrity checks in `Integrity.enforce()` ensures the following:

- length is not 0
- first condition has no parent
- _validateBFS(): parent of condition must be before condition and children are directly following their parents, ordered by indexes (BFS)
- _validateOperator(): calls `check...()` depending on the operator. Discards unsupported operators. Does differentiate between EqualTo and EqualToAvatar
- _validateEncoding(): if Encoding is Static, Dynamic, or EtherValue and operator is not Slice then there are no children
- _validateEncoding(): if Encoding is AbiEncoded, Tuple, or Array, then they have at least one structural child
- _validateVariantTypes(): if Operator is And, Or, or Array, loop through children to check that their hash matches in the TypeTree (i.e. they have the same hash or layout encoding is Dynamic, AbiEncoded, or None. They do not need to have the same encoding.)
- _validatePluckZipTypes(): for Operator ZipSome and ZipEvery, loop through children of only child, find plucked condition, resolve its layout, ensure that `_isTypeCompatible`

`(resolvedPluckedCondition.children[0], resolvedOnlyChildCondition.children[pluckedIndex])`.

- _validatePluckOrder(): if Operator is `WithinRatio`, the `referencePluckIndex` is `uint8(condition.compValue[0])` and `relativePluckIndex` is `uint8(condition.compValue[2])`, they must be visited after the Pluck condition. If Operator is `ZipSome`, or `ZipEvery`, the pluck index is `uint8(condition.compValue[k])`, it must be visited after the Pluck condition. The visit occur from `conditions[0]` and process its children in DFS order.

The individual checks performed for different operator are summed up in the following table.

Operator	paramType (encoding)	compValue	Other
`Pass`		empty	
`And`	None	empty	Must have children
`Or`	None	empty	Must have children
`Empty`	None	empty	No children
`Matches`	Tuple/Array/AbiEncoded	empty unless AbiEncoded	If AbiEncoded, compValue is 0 or 2 bytes or starts with 2 bytes encoded length (less than 24 bytes long). At least one child and all children structural
`ArraySome`	Array	empty	Exactly one child and it is structural
`ArrayEvery`	Array	empty	Exactly one child and it is structural
`ArrayTailMatches`	Array	empty	At least one child and all children structural

Operator	paramType (encoding)	compValue	Other
`ZipSome`/`ZipEvery`	None	at least 2 bytes	Exactly one child and it is structural, encoding is tuple, it is recovered as a tuple by `TypeTree.resolve(conditions, childStart)`, `childLayout.children.length == condition.compValue.length`. For each pluck index (defined by each byte of compValue) there is at least one condition with matching `compValue[0]` and it is encoded as Array. The only child (and its children) is not of operator `Pluck`
`Slice`	Static/Dynamic	3 bytes	Exactly 1 child and it has encoding Static
`Pluck`	Not None	1 byte and not 255	
`EqualToAvatar`	Static	empty	
`EqualTo`	Not None, not AbiEncoded	If encoding Static or EtherValue 32 bytes, if Array or Tuple < 32 bytes	
`Gt`,`Lt`,`SGt`,`SLt`	Static, EtherValue	32 bytes	
`BitMask`	Static, Dynamic	At least 4 bytes and even	

Operator	paramType (encoding)	compValue	Other
`Custom`		At least 20 bytes	
`WithinRatio`	None	12 bytes + 1 or 2 adapter blobs	No children, encoded ratios in compValue are not both 0
`WithinAllowance`	Static, EtherValue	32, 34, or >= 54 bytes	33rd and 34th bytes cannot be >27
`CallWithin Allowance`	None	32 bytes	No children

Status

The report is complete and has been sent to the core developer.

Legal Information And Disclaimer

1. This report is based solely on the information provided by Zodiac, with the assumption that the information provided to Gnosis is authentic, accurate, complete, and not misleading as of the date of this report. Gnosis has not conducted any independent enquiries, investigations or due diligence in respect of the Company, its business or its operations.
2. Changes to the information contained in the documents, repositories and any other materials referenced in this report might affect or change the analysis and conclusions presented. Gnosis is not responsible for monitoring, nor will we be aware of, any future additions, modifications, or deletions to the audited code. As such, Gnosis does not assume any responsibility to update any information, content or data contained in this report following the date of its publication.
3. This report does not address, nor should it be interpreted as addressing, any regulatory, tax or legal matters, including but not limited to: tax treatment, tax consequences, levies, duties, data privacy, data protection laws, issues relating to the licensing of information technology, intellectual property, money laundering and countering the financing of terrorism, or any other legal restrictions or prohibitions. Gnosis disclaims any liability for omissions or errors in the findings or conclusions presented in this report.
4. The views expressed in this report are solely our views regarding the specific issues discussed within this report. This report is not intended to be exhaustive, nor should it be construed as an assurance, guarantee or warranty that the code is free from bugs, vulnerabilities, defects or deficiencies. Different use cases may carry different risks, and integration with third-party applications may introduce additional risks.
5. This report is provided for informational purposes only and should not be used as the basis for making investment or financial decisions. This report does not constitute investment research and should not be viewed as an invitation, recommendation, solicitation or offer to subscribe for or purchase any securities, investments, products or services. Gnosis is not a financial advisor, and this report does not constitute financial or investment advice.
6. The statements in this report should be considered as a whole, and no individual statement should be extracted or referenced independently.
7. This report is addressed exclusively to the Company, and except with prior written consent from Gnosis, it may not be shared, disclosed, transmitted, or relied upon by any other person or entity for any purpose.
8. To the fullest extent permitted by applicable laws, Gnosis disclaims any and all other liability, whether in contract, tort, or otherwise, that may arise from this report or the use thereof.

Issues

[Med][Fixed] Authentication Signature Manipulation

Summary

`zodiac-core`'s `SignatureChecker.moduleTxSignedBy()` expects a calldata structured as `dataToBeHashed | 1271Signature | arbitrary | checkerSignature`. The whole calldata is not hashed and signed which allows setting up a malicious but hard to detect forgeable signature scheme.

Vulnerability Detail

The function `SignatureChecker.moduleTxSignedBy()` gets the `(v, r, s)` component of the signature as well as the `salt` from the end of the calldata. If `v` is zero, it attempts to verify the signature using an EIP1271 signature. It assumes the signature is pointed by `s` and computes the hash of what is signed by taking everything before `s`. This means that the expected structure of the calldata is `dataToBeHashed | 1271Signature | arbitrary | salt | v | r | s` with all parameters controlled by caller.

```
abstract contract SignatureChecker {
    ...
    function moduleTxSignedBy() internal view returns (bytes32, address) {
        bytes calldata data = msg.data;

        ...
        if (data.length < 4 + 32 + 65) {
            return (bytes32(0), address(0));
        }

        (uint8 v, bytes32 r, bytes32 s) = _splitSignature(data);

        uint256 end = data.length - (32 + 65);
        bytes32 salt = bytes32(data[end:]);

        ...
        if (v == 0) {
            uint256 start = uint256(s);
            if (start < 4 || start > end) {
                return (bytes32(0), address(0));
            }
            address signer = address(uint160(uint256(r)));

            bytes32 hash = moduleTxHash(data[:start], salt);
            return
                _isValidContractSignature(signer, hash, data[start:end])
                    ? (hash, signer)
```

```
                        : (bytes32(0), address(0));
            } else {
                ...
            }
        }
        ...
}
```

`Roles` execution functions call `_authenticate(roleKey)` which has the `moduleOnly` modifier which uses `moduleTxSignedBy()` to recover signer.

With `v = 0`, `r = authorizedModule`, and `s = craftedPointer` the hash sent to `r` is computed from the beginning part of the calldata until `s`. We can craft two different calldatas to `Roles.execTransactionWithRole()` that will result in the same hash being verified by `r`.

One of the exec function signature is:

```
    function execTransactionWithRole(
        address to,
        uint256 value,
        bytes calldata data,
        Operation operation,
        bytes32 roleKey,
        bool shouldRevert
    )
```

The following calldata (assume concatenation of listed elements) will result in a hash computed by `SignatureChecker.moduleTxSignedBy()` that does not include the actual `data` being sent during the underlying contract call:

- `execTransactionWithRole()` signature
- to = validContract
- value = 0
- data = pointer to encoded data location after 1271Signature (malicious)
- operation = 0
- roleKey = transferAuthorized
- shouldRevert = false
- 1271Signature expected by authorized module
- encodedData pointed by previous data pointer for valid call
- salt
- (v, r, s) = (0, authorizedModule, pointer to 1271Signature)

For this calldata, the hash will include the pointer of `data` but not the data itself as it will stop before `1271Signature`. If the signature is valid for one specific data, it will be valid for any data.

Impact

A malicious operator of a module multisig could propose accepting a signature to crafted calldata that looks like a legit call (e.g. a call to `function(arg1, arg2)` with signed data `function(arg1, pointerToArg2)`) but since the encoded data is outside the hashing range, the signature will also accept malicious calls (e.g. a call to `function(arg1, maliciousArg2)` with same pointer signed data). Depending on the toolchain, it may be almost impossible for the multisig signers to detect the malicious intent behind the signature.

If the `Roles` module is set up to accept the function call signed by the module multisig, both the valid and malicious transactions could be executed.

Code Snippets

https://github.com/gnosisguild/zodiac-core/blob/11708aca2f75a29a0a8a40be6aa8e8b925e8c9ad/contracts/signature/SignatureChecker.sol#L57

Recommendation

Hash and signature should be provided by consuming function to ensure proper encoding.

For example, replace:

```
function execTransactionWithRole(
        address to,
        uint256 value,
        bytes calldata data,
        Operation operation,
        bytes32 roleKey,
        bool shouldRevert
    ) public nonReentrant returns (bool success) {
        (address module, uint256 nextMembership) = _authenticate(roleKey);
        ...
    }
```

With:

```
function execTransactionWithRole(
        address to,
        uint256 value,
        bytes calldata data,
        Operation operation,
        bytes32 roleKey,
        bool shouldRevert,
        bytes32 salt,
        bytes signature,
    ) public nonReentrant returns (bool success) {
```

```
11          bytes32 hash = EIP712Encode(to, value, data, operation, roleKey,
                shouldRevert, salt);
12          (address module, uint256 nextMembership) = _authenticate(roleKey, hash,
                 signature);
13          ...
14      }
```

Where `EIP712Encode()` is similar to `SignatureChecker.moduleTxHash()`.

And let `_authenticate()`, `moduleOnly`, and `sentOrSignedByModule()` handle the verification with the given signature and hash without attempting to recover it from calldata themselves (as they cannot be aware of what should be in calldata and where).

Response

The code has been updated as of commit `0x172723b` to exactly sign the struct `ModuleTx({to: to, value: value, data: data, operation: operation})`. Unfortunately, this does not include the `roleKey` and `shouldRevert`. As a result, a signature for a module with a specific role could be applied to the same module with a different role.

This issue has low impact but the owner decided it needs to be fixed anyway.

This has been fixed in commit `0xa19001b`.

[Gas][Acknowledged] Membership._authenticate() verifies signatures twice

Summary

The function `Membership._authenticate()` used to verify the the call has been signed by a module (by appending a signature or being `msg.sender`) and that the membership of the module is valid verifies the signature twice from calldata which is a waste of gas.

Vulnerability Detail

The function `_authenticate()` uses the `moduleOnly` modifier before getting the valid signer via `sentOrSignedByModule()`

```
    function _authenticate(
        bytes32 roleKey
    ) internal moduleOnly returns (address module, uint256 nextMembership) {
        // Never authorize the zero role
        if (roleKey == 0) {
            revert NoMembership();
        }

        module = sentOrSignedByModule();
        ...
    }
```

Both the modifier and function call `SignatureChecker.moduleTxSignedBy()` to recover signer and validate signature in case `msg.sender` is not directly a module:

```
  modifier moduleOnly() {
    if (modules[msg.sender] == address(0)) {
      (bytes32 hash, address signer) = moduleTxSignedBy();

      ...
    }

    _;
  }

  function sentOrSignedByModule() internal view returns (address) {
    if (modules[msg.sender] != address(0)) {
      return msg.sender;
    }

    (, address signer) = moduleTxSignedBy();
    if (modules[signer] != address(0)) {
      return signer;
    }

    return address(0);
  }
```

Impact

Waste of gas by recovering twice the signature in case `msg.sender` is not a module.

Code Snippets

https://github.com/gnosisguild/zodiac-modifier-roles/blob/3c6f49d92128db20794774974f15b6c3b27dacf3/packages/evm/contracts/core/Membership.sol#L28

Recommendation

Get the signer and validate the signature is from a module once by inlining the logic of the modifier in the function.

Response

The issue has been acknowledged.

[Info][Answered] `Membership._hasActiveMembership()` does not take into account consumptions

Summary

The function `Membership._hasActiveMembership()` returns whether a module has an active membership for role key taking into account timestamps but not uses left. A module could have an active membership for a role key as seen from `Roles.isMember()` but fail to execute any transaction due to insufficient uses left.

Vulnerability Detail

The function `_hasActiveMembership()` does not check for uses left:

```
    function _hasActiveMembership(
        bytes32 roleKey,
        address module
    ) internal view returns (bool) {
        if (roleKey == 0) return false;
        uint256 membership = roles[roleKey].members[module];
        if (membership == 0) return false;
        if (block.timestamp < uint64(membership >> 192)) return false;
        if (block.timestamp > uint64(membership >> 128)) return false;
        return true;
    }
```

Impact

This could be intended behaviour.

Code Snippets

https://github.com/gnosisguild/zodiac-modifier-roles/blob/3c6f49d92128db20794774974f15b6c3b27dacf3/packages/evm/contracts/core/Membership.sol#L74

Response

Memberships are removed when uses left reach 0 so it cannot be that a membership has an active timestamp and no uses left.

[Info][Acknowledged] Targets with None clearance are not blocked

Summary

The inline documentation of `Setup` seem to state that targets with clearance `None` are blocked but implementation of `Authorization` falls back to using the selector to scope the config.

Vulnerability Detail

The inline documentation of `Setup` define `Clearance.None` as blocked target:

```
/*
 * Permission Model
 *
 * Role
 * │
 * ├─ members ──────────────────────► who can use this role
 * │
 * ├─ targets (address → Clearance)
 * │  │
 * │  ├─    Clearance.None ─────────► target blocked (default)  // @audit misleading
 *  ...
 */
```

The function that verifies the transaction can be executed uses the function selector when clearance is `None` instead of blocking the call:

```
    function _transaction(
        ...
    ) private view returns (Consumption[] memory) {
        ...
        uint256 scopeConfig;
        if (clearance != Clearance.None) {
            ...
        }

        // Global entry
        if (scopeConfig == 0 && data.length != 0) {
            scopeConfig = role.scopeConfig[bytes32(bytes4(data)) >> 160];
        }

        if (scopeConfig == 0) {
            revert TransactionNotAllowed(transaction.to, bytes4(data));
        }
        ...
    }
```

Impact

If an owner calls `Setup.revokeTarget()` to revoke a target which sets the clearance to `None` expecting calls to be blocked, transactions could still be executed if the default config allows the selector.

This may be intended behaviour.

Code Snippets

https://github.com/gnosisguild/zodiac-modifier-roles/blob/3c6f49d92128db20794774974f15b6c3b27dacf3/packages/evm/contracts/core/Setup.sol#L22

https://github.com/gnosisguild/zodiac-modifier-roles/blob/3c6f49d92128db20794774974f15b6c3b27dacf3/packages/evm/contracts/core/Authorization.sol#L91

Response

The issue has been acknowledged.

[Info][Acknowledged] `ConditionEvaluator.evaluate()` misinterprets placeholders

Summary

The function `ConditionEvaluator.evaluate()` does a logic switch on the value of `operator` but interprets `EqualToAvatar` (that should not exist in this place of the code due to being replaced with `EqualTo` by ConditionUnpacker) and `3`, `9`, `12` placeholders as `zipIterators`. It also interprets remaining operator placeholders as custom conditions.

Incorrectly encoded conditions with invalid operators will interpret the condition arbitrarily instead of reverting / returning a violation which may cause confusion.

Vulnerability Detail

Impact

No expected impact.

Code Snippets

https://github.com/gnosisguild/zodiac-modifier-roles/blob/3c6f49d92128db20794774974f15b6c3b27dacf3/packages/evm/contracts/core/evaluate/ConditionEvaluator.sol

Recommendation

Revert or return a violation when unexpected operator are used.

Response

The issue has been acknowledged.

[Info][Fixed] Misleading documentation of `Integrity._checkSlice()`

Summary

The inline documenation of `Integrity._checkSlice()` states that the condition must have at most 1 child and if it is structural it must be Static. The code enforces that there is exactly one child and that is Static. I think the code makes more sense than the inline documentation.

Vulnerability Detail

```
    function _checkSlice(
        ConditionFlat[] memory conditions,
        uint256 index
    ) private pure {
        ConditionFlat memory condition = conditions[index];
        ...

        (uint256 childStart, uint256 childCount) = Topology.childBounds(
            conditions,
            index
        );
        // Children: At most 1 child  // @audit cannot be 0
        if (childCount != 1) {
            revert IRolesError.UnsuitableChildCount(index);
        }

        // If it has a structural child, it must resolve to Static  // @audit
            does not check structural
        if (conditions[childStart].paramType != Encoding.Static) {
            revert IRolesError.SliceChildNotStatic(index);
        }
    }
```

Impact

Misleading documentation.

Code Snippets

https://github.com/gnosisguild/zodiac-modifier-roles/blob/3c6f49d92128db20794774974f15b6c3b27dacf3/packages/evm/contracts/core/serialize/Integrity.sol#L367

Recommendation

Confirm code behaviour is intended and update documentation.

Response

Updated the inline documentation in commit `0xe808a44`

[Info][Fixed] Misleading documentation of `Integrity._checkPluck()`

Summary

The inline documentation of `Integrity._checkPluck()` states the ParamType should be Static / EtherValue / Array while it also allows Dynamic / Tuple / AbiEncoded.

Vulnerability Detail

```
    function _checkPluck(
        ConditionFlat[] memory conditions,
        uint256 index
    ) private pure {
        ConditionFlat memory condition = conditions[index];
        Encoding encoding = condition.paramType;
        // ParamType: Static / EtherValue / Array  // @audit can be  Dynamic,
            Tuple, AbiEncoded
        if (encoding == Encoding.None) {
            revert IRolesError.UnsuitableParameterType(index);
        }
        // CompValue: 1 byte
        if (
            condition.compValue.length != 1 ||
            uint8(condition.compValue[0]) == 255
        ) {
            revert IRolesError.UnsuitableCompValue(index);
        }
    }
```

Impact

Misleading documentation.

Code Snippets

https://github.com/gnosisguild/zodiac-modifier-roles/blob/3c6f49d92128db20794774974f15b6c3b27dacf3/packages/evm/contracts/core/serialize/Integrity.sol#L401

Recommendation

Confirm code behaviour is intended and update documentation.

Response

Updated the inline documentation in commit `0xe808a44`

[Info][Fixed] Topology.inlinedSize() reverts on non inlined element

Summary

The function `Topology.inlinedSize()` documents that it would return 0 if input is not inlined but it reverts in that case instead.

When the element is inlined and a tuple, the function loops through children to return the sum of the size of its elements. There seem to be no reason for some children of the tuple to not be inlined causing a revert.

Vulnerability Detail

```
1     /**
2      * @notice Computes inlined size for a node (0 if not inlined)  // @audit
           not true
3      */
4     function inlinedSize(
5         ConditionFlat[] memory conditions,
6         uint256 index
7     ) internal pure returns (uint256 size) {
8         assert(isInlined(conditions, index) == true);
9
10        ...
11        Encoding encoding = conditions[index].paramType;
12
13        if (encoding == Encoding.Static) {
14            return 32;
15        }
16
17        ...
18        (uint256 childStart, uint256 childCount) = childBounds(
19            conditions,
20            index
21        );
22
23        for (uint256 i; i < childCount; ++i) {
24            uint256 childSize = inlinedSize(conditions, childStart + i);
25            ...
26            if (encoding == Encoding.None && childSize > 0) return childSize;
27            size += childSize;
28        }
29    }
```

Impact

Function `inlinedSize()` may revert unexpectedly. It is used during packing of conditions so should not have a high impact.

Code Snippets

https://github.com/gnosisguild/zodiac-modifier-roles/blob/6a3a46862bd730d6a7817b5845b0d0002ebaf871/packages/evm/contracts/core/serialize/Topology.sol#L71

Recommendation

Ensure code behaviour is intended and update documentation

Response

`Topology.inlinedSize()` is only called when the whole topology of the conditions is inlined in `ConditionPacker._packNodes()`.

Updated the inline documentation in commit `0xe808a44`

[Question][Answered] `ConditionEvaluator.__input()` value may unexpectedly be a hash

Summary

The function to get an input value from calldata `ConditionEvaluator.__input()` returns a hash of part of the calldata in case the size e.g. determined by `AbiLocation.size(data, location, condition)` is greater than 32. Otherwise it returns a value loaded from context or location.

To my understanding, a condition might expect for example a fixed size ABI encoded value within a certain allowance but receives a hash of encoded data longer than 32 bytes instead. The size of the encoded data depends on the user provided calldata and not on the owner setting up the condition.

Vulnerability Detail

```
    function __input(
        bytes calldata data,
        uint256 location,
        Condition memory condition,
        Context memory context
    ) private pure returns (bytes32 result, bool overflow) {
        ...
        // Check if condition has size set (e.g., from Slice), otherwise get
            from decoder
        uint256 size = condition.size != 0
            ? condition.size
            : AbiLocation.size(data, location, condition);

        ...

        if (size == 32) {
            assembly {
                result := calldataload(add(data.offset, location))
            }
            return (result, false);
        }

        ...

        // size > 32
        return (keccak256(data[location:location + size]), false);
    }
```

Impact

I am unsure this can occur in practice that a condition expects a fixed size ABI encoded data to decode it as `__input()` and pass with an hash instead. This functions is used for operators EqualTo, GreaterThan, LessThan, SignedIntGreaterThan, SignedIntLessThan, WithinAllowance or within `_pluck()`.

Code Snippets

https://github.com/gnosisguild/zodiac-modifier-roles/blob/3c6f49d92128db20794774974f15b6c3b27dacf3/packages/evm/contracts/core/evaluate/ConditionEvaluator.sol#L570

Response

It has been discussed wether the data should be always hashed to have a consistent behaviour.

> `__input()` is a shared utility. Operator comparisons like Gt, Lt, and signed variants require the raw value, so hashing would break them. Integrity constrains those callers to Static or EtherValue, so they are structurally excluded from the hash branch.

> EqualTo is the only operator that can reach either branch (hash non hash). There yes we are hashing only when larger than 32. This is the only case where both behaviors exist we could perhaps adjust behavior.

[Question][Answered] Adding 32 bytes to start location

Summary

The function `BitmaskChecker.check()` that applies a bitmask to compare calldata bytes to expected values adds 32 bytes to the starting point of the comparison in case it is not inlined. I do not understand the reason.

`ConditionEvaluator.evaluate()` also adds 32 bytes to the location for Operator.Matches if encoding is AbiEncoded, which may be related.

Vulnerability Detail

```
library BitmaskChecker {
    function check(
        bytes calldata data,
        uint256 location,
        bytes memory compValue,
        bool inlined
    ) internal pure returns (Status) {
        uint256 shift = uint16(bytes2(compValue));
        uint256 length = (compValue.length - 2) / 2;

        uint256 start = location + (inlined ? 0 : 32); // @audit idk why we add
            32
        uint256 end = data.length;

        ...
    }
}
```

```
library ConditionEvaluator {
    function evaluate(
        ...
    ) internal view returns (Result memory) {
        Operator operator = condition.operator;

        if (operator < Operator.EqualTo) {
            if (operator == Operator.Pass) {
                ...
            } else if (operator == Operator.Matches) {
                location = location > 0 &&
                    condition.encoding == Encoding.AbiEncoded
                    ? location + 32  // @audit idk
                    : location;
                return
                    _matches(data, location, condition, consumptions, context);
            }
            ...
        }
```

```
20         }
21     }
```

Code Snippets

https://github.com/gnosisguild/zodiac-modifier-roles/blob/contracts-v3/packages/evm/contracts/core/evaluate/BitmaskChecker.sol

Response

Dynamic types in the code are really just bytes and strings. Dynamic structs will be encoded as tuple. Static-size array are encoded as tuples and dynamic-size array as Arrays. In which case the first word of dynamic types is always the length of the encoded element.

Bitmask only applies to static and dynamic as defined above (not dynamic structs, i.e. buffers).

A new commit `0xa28ac04` has been added to tighten the boundary checks in `BitmaskChecker`

[Question][Answered] AbiLocation improper use of dynamic elements

Summary

The library `AbiLocation` documents and behaves as if the first word of encoded dynamic elements is the size of the element which is not the case for structs containing dynamic elements such as bytes.

Vulnerability Detail

The function `size()` that computes the size of an ABI encoded element at location in data assumes the first word is the size of the element:

```
    function size(
        bytes calldata data,
        uint256 location,
        Condition memory condition
    ) internal pure returns (uint256 result) {
        // Every ABI-encoded element occupies at least one word.
        if (location + 32 > data.length) return data.length + 1;

        Encoding encoding = condition.encoding;
        if (encoding == Encoding.Static) {
            return 32;
        }

        /*
         * Always preload the first word - its meaning depends on encoding:
         * - Dynamic: byte length of content  // @audit not true for dynamic structs
         * - Array: element count
         */
        uint256 word;
        assembly {
            word := calldataload(add(data.offset, location))
        }

        ...

        if (encoding == Encoding.Dynamic || encoding == Encoding.AbiEncoded) {
            // Dynamic types: length prefix + padded content (ceil32)
            return 32 + ((word + 31) & ~uint256(31));
        }
    }
```

Suppose a struct and function defined as:

```
struct myStruct {
    uint256 x,
    bytes y,
```

```
    uint256 z
}

function test(uint256 a, myStruct b);
```

A call to `test(123456, [1111,"0x1234",2222])` will have the following encoding:

```
89202f61                                                                 // selector
000000000000000000000000000000000000000000000000000000000001e240  // 123456
0000000000000000000000000000000000000000000000000000000000000040  // where to find the struct relative to start of encoded data
0000000000000000000000000000000000000000000000000000000000000457  // 1111
0000000000000000000000000000000000000000000000000000000000000060  // where to find "0x1234" relative to start of struct
00000000000000000000000000000000000000000000000000000000000008ae  // 2222
0000000000000000000000000000000000000000000000000000000000000002  // length of "0x1234"
1234000000000000000000000000000000000000000000000000000000000000  // "0x1234"
```

The first word of the struct at position `0x40` is not the length of the struct but the first element of the struct.

Impact

No idea, maybe it is expected that the encoding is different.

This also impacts the `_tailLocation()` function that could point at anything since it is used with the first word of dynamic elements:

```
    function _tailLocation(
        bytes calldata data,
        uint256 location,
        uint256 headOffset
    ) private pure returns (uint256) {
        ...

        uint256 tailOffset;
        assembly {
            tailOffset := calldataload(
                add(data.offset, add(location, headOffset))  // @audit can point at anything
            )
        }

        ...
    }
```

Code Snippets

https://github.com/gnosisguild/zodiac-modifier-roles/blob/3c6f49d92128db20794774974f15b6c3b27dacf3/packages/evm/contracts/common/AbiLocation.sol#L101

Response

Dynamic structs are encoded as tuples and not `AbiEncoded`.

[Question][Answered] AbiLocation when encoding is None takes first element

Summary

I am sure it makes sense but I cannot figure out why AbiLocation.size() only takes the size of the first valid children and ignores invalid values.

Vulnerability Detail

```
    function size(
        bytes calldata data,
        uint256 location,
        Condition memory condition
    ) internal pure returns (uint256 result) {
        ...

        if (encoding == Encoding.None) {
            /*
             * Logical nodes (And/Or) are transparent. We delegate the size
             * calculation to the first valid (non-empty, non-overflowing)
             * child subtree.
             *
             * This works for both:
             * - Non-Variant: All children share the same type tree.
             * - Variant: Children have different types, but variants are
             *            always Dynamic-encoded, so size is deterministic.
             */
            for (uint256 i; i < childCount; ++i) {  // @audit idk why we get
                the size of the first valid element
                result = size(data, location, condition.children[i]);
                if (result > 0 && result <= data.length) return result;
            }  // @audit ignores subsequent overflows
            return data.length + 1;
        }
        ...
    }
```

The logic is similar in `Topology.inlinedSize()`:

```
library Topology {
    function inlinedSize(
        ConditionFlat[] memory conditions,
        uint256 index
    ) internal pure returns (uint256 size) {
        ...

        for (uint256 i; i < childCount; ++i) {
            uint256 childSize = inlinedSize(conditions, childStart + i);
            /*
             * None nodes are transparent wrappers (And/Or or non-structural).
             * Since inlined nodes can't have variants, all structural children
```

```
13                 * share the same size. Return on first non-zero (structural) child
                       .
14                 */
15                if (encoding == Encoding.None && childSize > 0) return childSize;
16                size += childSize;
17            }
18        }
```

Response

None with structural children are And/Or. For these, elements are either a tree of stuff that matches (everything is the same) as verified by `Integrity._validateVariantTypes()`, or they are stored within a bytes which all use 32 bytes length.